

Advanced Manual

Smart Contract Audit

September 5, 2026

✂ x.com/coinsultaudits

📌 t.me/coinsult_tg

Audit requested by

PresaleSale

0x319c27ba4C3248baD05d67F083C6a5f593A2D517

Global Overview

Manual Code Review

In this audit report we will highlight the following issues:

Vulnerability Level	Total	Pending	Acknowledged	Resolved
● Informational	3	0	3	0
● Low-Risk	6	2	4	0
● Medium-Risk	2	0	2	0
● Critical-Risk	0	0	0	0

Risk Classification

Coinsult uses certain vulnerability levels, these indicate how bad a certain issue is. The higher the risk, the more strictly it is recommended to correct the error before using the contract.

Vulnerability Level	Description
● Informational	Does not compromise the functionality of the contract in any way
● Low-Risk	Won't cause any problems, but can be adjusted for improvement
● Medium-Risk	Will likely cause problems and it is recommended to adjust
● Critical-Risk	Will definitely cause problems, this needs to be adjusted

Audit Summary

PresaleSale

Website	http://roodrai.org
Blockchain	Ethereum mainnet
Source Code	https://etherscan.io/address/0x319c27ba4C3248baD05d67F083C6a5f593A2D517#code
Contract Address	0x319c27ba4C3248baD05d67F083C6a5f593A2D517

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Audit Scope

Coinsult was commissioned to perform an audit based on the provided code.

Note that we only audited the code available to us on this URL at the time of the audit. If the URL is not from any block explorer (main net), it may be subject to change. Always check the contract address on this audit report and compare it to the token you are doing research for.

Audit Method

Coinsult's manual smart contract audit is an extensive methodical examination and analysis of the smart contract's code that is used to interact with the blockchain. This process is conducted to discover errors, issues and security vulnerabilities in the code in order to suggest improvements and ways to fix them.

Automated Vulnerability Check

Coinsult uses software that checks for common vulnerability issues within smart contracts. We use automated tools that scan the contract for security vulnerabilities such as integer-overflow, integer-underflow, out-of-gas-situations, unchecked transfers, etc.

Manual Code Review

Coinsult's manual code review involves a human looking at source code, line by line, to find vulnerabilities. Manual code review helps to clarify the context of coding decisions. Automated tools are faster but they cannot take the developer's intentions and general business logic into consideration.

Used tools

- Slither: Solidity static analysis framework
- Remix: IDE Developer Tool
- CWE: Common Weakness Enumeration
- SWC: Smart Contract Weakness Classification and Test Cases
- DEX: Testnet Blockchains

Table of Contents

Global Overview	2
Manual Code Review	2
Audit Summary	2
Audit Scope	3
Audit Method	3
Automated Vulnerability Check	3
Manual Code Review	3
Used tools	3
Table of Contents	4

● Forced Native Currency Sent To The Contract Can Never Be Recovered

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
PRE-11	Status: Acknowledged by the client as intended behaviour. The client notes that real payments are always forwarded immediately and never rest in the contract, so this affects only funds sent by accident or forced in outside normal operation. We agree with that reading, and the deployed contract's balance of zero is consistent with it. Plain transfers are refused outright, which is the right decision and which we are not asking to change. The residual is limited to amounts arriving by contract self-destruction or block rewards, which cannot be swept and are typically negligible. No buyer funds are exposed.

Code Snippet

```
// plain transfers are refused, which is correct
receive() external payable { revert("Use buyWithEth()"); }

// tokens can be rescued, native currency cannot:
function rescueStuckTokens(address token, address to, uint256 amount)
```

Recommendation

Accepted as an operational trade-off. No change required. If a future version is deployed, an owner-only sweep of the native balance would close the gap at very little risk, since real payments never rest in the contract.

● There Is No On-Chain Way To Mark The Sale As Finished

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
PRE-I2	Status: Acknowledged by the client, with pausing adopted as the closing mechanism and to be formalised as an explicit step in the launch runbook. We accept this. The one point worth keeping visible is that pausing is reversible and is controlled by the same single key discussed under the owner-control finding, so a paused sale is a statement of intent rather than a guarantee that the sale has ended. Buyers cannot confirm from the contract alone that no further purchases will ever be accepted.

Code Snippet

```
// the counter stops at the last stage but selling continues
if (next >= TOTAL_STAGES) revert AlreadyAtFinalStage();

// pause() is the closing mechanism, and unpause() can undo it
```

Recommendation

Accepted. Make the closing pause an explicit, signed-off runbook step as planned, and state clearly in buyer-facing material that the sale is closed operationally rather than permanently in code.

● Small Housekeeping Points Around Token Listing And Rescue

Informational - Does not compromise the functionality of the contract in any way [**Status: Acknowledged**]

Error Code	Description
PRE-I3	Status: Acknowledged by the client as owner-only configuration actions that do not affect buyer funds. We agree. Removing a token from the approved list leaves its recorded decimals behind, the rescue function can send recovered tokens to any address rather than only the treasury, and approving a token reads its decimals with no upper limit check. None of these is reachable by anyone other than the owner and none touches recorded contributions. They remain worth tidying in any future version, and the rescue destination in particular is one more thing a compromised owner key could use.

Code Snippet

```
// de-listing leaves the cached decimals behind
allowedToken[token] = allowed;
if (allowed) { tokenDecimals[token] = IERC20Metadata(token).decimals(); }

// rescue may send anywhere, not only to the treasury
IERC20(token).safeTransfer(to, amount);
```

Recommendation

Accepted for this deployment. In any future version, clear the recorded decimals on removal, restrict rescued tokens to the treasury, and reject an implausible decimals value on approval.

● Very Small Payments Can Be Taken While Recording Nothing

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PRE-L1	Status: Acknowledged by the client as immaterial at the sums involved. We accept the conclusion, with one correction to the reasoning. The client cites minimum-buy enforcement in the front end, but the contract is open to direct calls, so anyone can bypass the front end entirely and the front-end minimum does not constrain on-chain behaviour. The reason this is acceptable is immateriality rather than front-end enforcement: the amounts that round to nothing are vanishingly small. The residual is unchanged. A payment small enough to round down records no contribution while the tokens are still taken, and the sender is still added to the unique-buyer count, so that figure can be inflated cheaply by anyone calling the contract directly. It is a bookkeeping matter, not a material loss.

Code Snippet

```
// remainder is discarded, and zero is never rejected
return amount / (10 ** (decimals - USD_DECIMALS));

// still counted as a buyer even when the value rounds to zero
if (!_hasPurchased[buyer]) { totalBuyers += 1; }
```

Recommendation

Accepted for this deployment. Treat the unique-buyer figure as indicative rather than exact when reporting it publicly, since it can be inflated by direct calls. A rejection of zero-valued purchases would close it in any future version.

● Approved Stablecoins Are Always Counted As Exactly One Dollar

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PRE-L2	Status: Acknowledged by the client as an intended simplification for well-established stablecoins, with de-peg events to be monitored operationally. We accept this as a reasonable trade-off. The exposure is unchanged: if an approved coin loses its peg, buyers can pay with a discounted coin and receive full credit, and the loss falls on the project rather than the buyer. The safeguards are the owner-controlled approval list and the pause function, both of which depend on somebody noticing quickly. In past de-peg events the largest discounts appeared within the first hours, so the value of this mitigation rests on monitoring being genuinely prompt rather than on periodic review.

Code Snippet

```
// approved tokens are converted by decimals only, never by price
uint8 decimals = tokenDecimals[token];
// no oracle, no peg check
```

Recommendation

Accepted. Put concrete de-peg alerting in place for every approved coin with a threshold and a named responder, so that pausing or de-listing can happen in minutes rather than after the fact.

● The Sequencer Health Check Accepts An Incomplete Round

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PRE-L3	Status: Acknowledged by the client as applying only to the layer-two deployment. Confirmed correct for the mainnet contract. We verified that the sequencer uptime feed is set to the blank address on this deployment, so the check returns immediately and this finding is dormant here. It remains live for the layer-two deployment, which we have not been given an address for and have therefore not reviewed. Two points carry across to it. The missing check is that a reported start time of zero, meaning an incomplete round, passes the elapsed-time comparison when it should not. And because the price feed setter has been removed from this version, the sequencer feed address is now fixed permanently at deployment on that chain too, so it must be correct at launch.

Code Snippet

```
// dormant on mainnet: sequencerUptimeFeed == address(0), returns early
if (address(sequencerUptimeFeed) == address(0)) { return; }

// live on the layer-two deployment:
if (block.timestamp - startedAt <= sequencerGracePeriod) { ... }
// startedAt == 0 means an incomplete round, and passes this check
```

Recommendation

Accepted for the mainnet contract, where it cannot be reached. For the layer-two deployment, confirm the sequencer feed address and a non-zero grace period are

correct before launch, since neither can be changed afterwards. We are happy to review that deployment separately if you send the address.

● A Token That Takes A Cut On Transfer Would Be Credited In Full

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Acknowledged**]

Error Code	Description
PRE-L4	Status: Acknowledged by the client on the basis that only stablecoins without transfer fees are approved. We accept the approach, with one caveat that should stay on the record. The contract's own comments name a set of candidate stablecoins, and at least one widely used stablecoin in that category carries a dormant transfer-fee mechanism that its issuer could switch on without any change on your side. So the rule holds today, but it is a statement about current behaviour rather than a structural guarantee, and it needs periodic re-checking rather than a one-time check at approval. If it were ever switched on, buyers would be credited in full while the treasury received less, and the shortfall would fall on the project.

Code Snippet

```
// credited first, in full
uint256 usdValue = _tokenToUsd(token, amount);
_recordPurchase(msg.sender, token, amount, usdValue);

// then moved, with no check on what actually arrived
IERC20(token).safeTransferFrom(msg.sender, treasury, amount);
```

Recommendation

Accepted. Record the no-transfer-fee requirement as a standing check rather than a one-off, including for dormant mechanisms that are currently switched off, and re-verify it periodically for each approved coin while the sale is running.

● The Price Feed Addresses Are Now Fixed Permanently With No Way To Change Them

Low-Risk - Won't cause any problems, but can be adjusted for improvement [**Status: Pending**]

Error Code	Description
PRE-L5	Status: New in this review, introduced by the redeployment. The function that allowed the price feed addresses to be changed has been removed. We verified against the deployed contract that it is genuinely absent, and that the only remaining configuration functions are the ones for approved tokens, oracle parameters, the stage and the treasury. We understand this was done to close off a scaling defect on the backup feed path, and it achieves that: with no backup feed configured and no way to add one, that path can never run. The cost is that all three feed addresses are now fixed forever at their deployment values, and this deployment has no backup feed configured. If the main price feed is ever retired, replaced or starts misbehaving, purchases in the native currency stop permanently and there is no way to point the contract at a working feed. The only remedy would be deploying a new contract, which splits the record of contributions across two addresses. Purchases in approved tokens are unaffected, since they do not use a feed. The likelihood is low: the feed in use is the long-standing main-network reference for this pair and its address has been stable for years. The point is that the recovery option has been removed rather than that failure is expected.

Code Snippet

```
// verified against the deployed contract: no setPriceFeeds in the ABI.
// remaining setters: setAllowedToken, setOracleParams, setStage,
//                      setTreasury

// therefore fixed for the life of the contract:
primaryPriceFeed    = 0x5f4eC3Df9cbd43714FE2740f5E3616155c5b8419
secondaryPriceFeed  = 0x0000000000000000000000000000000000000000000000000000000000000000
sequencerUptimeFeed = 0x0000000000000000000000000000000000000000000000000000000000000000
```

Recommendation

Accept this consciously rather than by default. Removing the setter was a reasonable way to close the scaling defect, but it trades a configuration risk for an availability risk with no recovery path. Monitor the feed for staleness and for any deprecation notice from the provider, and have a documented plan for what you would do if it failed, including how contributions recorded here would be combined with those on a replacement contract.

One point to carry into any future version, because it is easy to lose: the conversion itself was never corrected. The code still reads the number of decimal places from the main feed even when the price came from the backup, so redeploying this source with a backup feed configured would reintroduce a serious mispricing fault. Fix that conversion, ideally by recording both feeds' decimals when they are configured and rejecting a backup whose scale differs, and then the ability to change feeds can safely be restored alongside it.

● The Staleness Limit Exactly Equals The Feed's Update Interval, Leaving No Margin

Low-Risk - Won't cause any problems, but can be adjusted for improvement [Status: Pending]

Error Code	Description
PRE-L6	Status: New in this review, from the deployed configuration. A price is rejected as out of date once it is older than the configured staleness limit, which is set to 3,600 seconds. The feed in use publishes a fresh answer either when the price moves beyond a small threshold or, failing that, once every 3,600 seconds. The limit therefore equals the feed's own maximum update interval exactly, with no margin at all. In quiet markets, where the price does not move enough to trigger an early update, the feed runs the full hour between answers. Any purchase attempted in the window after the limit expires but before the next answer is published is rejected. Because this deployment has no backup feed and the feed addresses can no longer be changed, that rejection is outright: the purchase simply fails. No funds are at risk and buyers can retry, so this is an availability and experience problem rather than a safety one. It is likely to appear as intermittent, unexplained purchase failures concentrated in calm market conditions, which is exactly when it is hardest to diagnose.

Code Snippet

```
// deployed configuration, verified on-chain:
maxPriceStaleness = 3600 // seconds

// the feed's published update interval is also 3600 seconds,
// so the margin is zero.
```

```
// on expiry the answer is treated as unusable:
if (block.timestamp - updatedAt > maxPriceStaleness) return 0;
// with no backup feed configured, this then reverts:
if (address(secondaryPriceFeed) == address(0)) revert OracleUnavailable();
```

Recommendation

Raise the staleness limit so it comfortably exceeds the feed's update interval, for example to between 4,500 and 7,200 seconds, which keeps genuinely stale prices out while absorbing normal timing variation. This can be done immediately on the live contract through the oracle parameter function and needs no redeployment, which is worth doing before the sale opens. Confirm the feed's current published update interval directly with the provider rather than relying on this report, and monitor for failed purchases so the setting can be revisited if failures still appear.

● The Owner Controls The Destination Of All Funds, With Immediate Effect

Medium-Risk - Will likely cause problems and it is recommended to adjust [Status: Acknowledged]

Error Code	Description
PRE-M1	Status: Acknowledged by the client, who confirms ownership currently sits with a single wallet rather than a multi-signature setup, with migration tracked as a near-term operational item. We record the acknowledgement, and we are leaving the severity where it is. Acknowledging a risk does not reduce it, and on the evidence this is the largest remaining risk in the deployment. We verified that the owner is the deploying account and that it can change the address all incoming funds are forwarded to, in one transaction, with immediate effect and no delay or announcement. A single compromised key is therefore sufficient to redirect every payment made after that point. The contract's own comments already state that the funds destination is expected to be a multi-signature wallet rather than an ordinary one. One part of this has improved. Removing the price feed setter means the owner can no longer replace the feeds, so the ability to corrupt recorded contributions through a chosen feed is gone. The control over where money goes is unchanged.

Code Snippet

```
// verified: owner is the deploying account, a single wallet
// verified: still present and immediate, with no delay
function setTreasury(address newTreasury) external onlyOwner {
    treasury = newTreasury;
}

// no longer present in the deployed contract (improvement):
// function setPriceFeeds(...)
```

Recommendation

Complete the multi-signature migration before the sale takes significant funds, not after. Until it is done, this single key is the main thing standing between the sale and its proceeds. Transfer ownership using the two-step process the contract already supports, monitor and alert on any treasury change, and state the current custody arrangement plainly in buyer-facing material so participants can judge it for themselves.

● Buyers Cannot Protect Themselves Against The Stage Moving Before Their Payment Lands

Medium-Risk - Will likely cause problems and it is recommended to adjust [**Status: Acknowledged**]

Error Code	Description
PRE-M2	Status: Acknowledged by the client as an accepted design trade-off, with commitments not to advance stages during high-volume purchase windows and to announce stage changes in advance. We record the acknowledgement and the mitigations, which are sensible and address the realistic case. The severity stays as it is, because the underlying property is unchanged: a purchase is recorded against whichever stage is active when it is included, buyers cannot state what they are agreeing to, and because stages only ever get more expensive, a change while a payment is in flight can only work against the buyer. The mitigations are operational rather than enforced by the contract, so they rest on the same single key discussed above and cannot be verified by a buyer or by us. That does not make them worthless, but it does mean the protection is a commitment rather than a guarantee, and it should be described that way if it is mentioned publicly.

Code Snippet

```
// whatever is current when the transaction lands, is what applies
uint8 stage = currentStage;

// no buyer-supplied expectations on either call
function buyWithEth() external payable
function buyWithToken(address token, uint256 amount) external
```

Recommendation

Accepted. Keep the commitments and make them concrete: announce stage changes with a fixed lead time, publish them where buyers will see them, and avoid advancing during busy periods as stated. In any future version, letting buyers pass the stage they expect would turn the commitment into something the contract enforces.

Disclaimer

This audit report has been prepared by Coinsult's experts at the request of the client. In this audit, the results of the static analysis and the manual code review will be presented. The purpose of the audit is to see if the functions work as intended, and to identify potential security issues within the smart contract.

The information in this report should be used to understand the risks associated with the smart contract. This report can be used as a guide for the development team on how the contract could possibly be improved by remediating the issues that were identified.

Coinsult is not responsible if a project turns out to be a scam, rug-pull or honeypot. We only provide a detailed analysis for your own research.

Coinsult is not responsible for any financial losses. Nothing in this contract audit is financial advice, please do your own research.

The information provided in this audit is for informational purposes only and should not be considered investment advice. Coinsult does not endorse, recommend, support or suggest to invest in any project.

Coinsult can not be held responsible for when a project turns out to be a rug-pull, honeypot or scam.